

Heuristic Functions

- An **admissible heuristic** never overestimates the cost of reaching the goal state;
 - $h(s)$ is **admissible** if, $\forall s \in S$:
 - $h(s) \leq h^*(s)$, where $h^*(s)$ is the **optimal** cost to achieve the goal.
- A **non-admissible heuristic** overestimates the cost to achieve the goal state;
- For example:
 - $h^*(Rome, Naples) = 200km$, the **optimal** cost from *Rome* to *Naples*.
 - $h_1(Rome, Naples) = 150km$, h_1 is **admissible**;
 - $h_1(Rome, Naples) = 199km$, h_1 is **admissible**;
 - $h_2(Rome, Naples) = 201km$, h_2 is **non-admissible**;
 - $h_2(Rome, Naples) = 300km$, h_2 is **non-admissible**;

- How to design informative (near optimal, admissible) heuristics?
 - **Domain-Dependent Heuristics** are usually “easier” to design;
 - **Domain-Independent Heuristics** are “more difficult” to design;

Heuristic Search: Direction

- How to apply *Heuristic Search* to solve a problem?
It depends on the **search direction**.
 - **Progressive Search:** *Forward* from initial state s_0 to a goal state G ;
 - **Regressive Search:** *Backward* from goal state G to initial state s_0 ;
 - **Bidirectional Search:** Searching *forward* and *backward* to “meet” in the middle;

In this lecture, we study **Progressive (Forward) Heuristic Search**.

Deterministic Heuristic Search (A^*)

A* Search

- A* is one of the most famous search algorithms in the literature³;
- A* performs **heuristic search by evaluating states** using f :
 - $f(s) = h(s) + g(s)$
 - $h(s)$ **estimated cost** to G from s ;
 - $g(s)$ **cost** from the initial state s_0 to s .
- $f(s) =$ **estimated cost of the cheapest plan through s .**
- **Input:** a state-space search problem;
- **Output:** a solution (a plan), or failure;

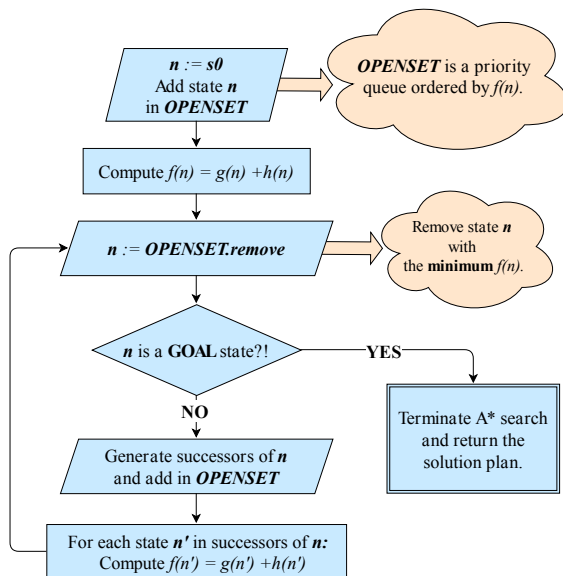
³ Hart, P. E.; Nilsson, N. J.; Raphael, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*. 1968.

Assumptions of A* Search

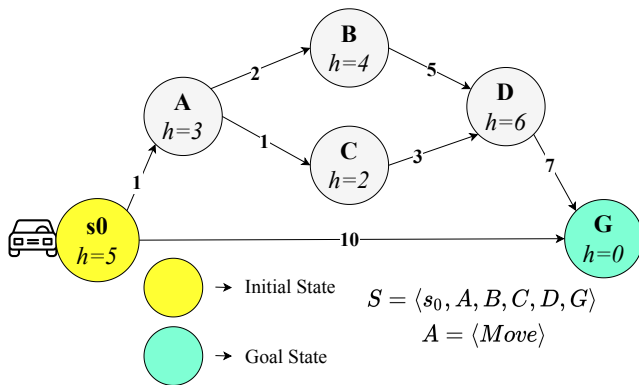
- **Single-agent**⁴;
- The states are **fully-observable** of the environment are known;
- **Finite number** of states S and actions A ;
- Actions have **deterministic effects**;

⁴ A* can be adapted to multiple agents.

A* Search Algorithm

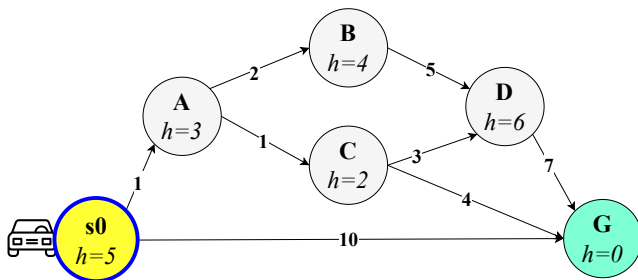


A* Search Example (1 of 5)



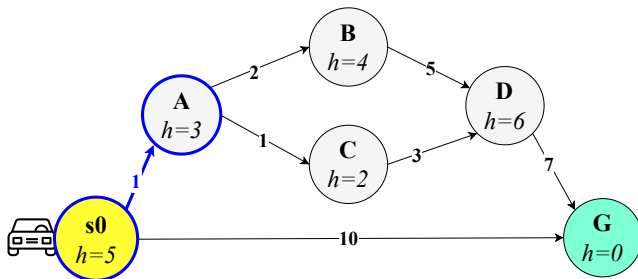
A* Search Example (2 of 5)

- $OPENSET = \langle s_0 \rangle$
 - $f(s_0) = g(s_0) + h(s_0) = 0 + 5 = 5$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: s_0
 - **Add successors** of s_0 in $OPENSET$:
 - $OPENSET = \langle \underline{A}, \underline{G} \rangle$



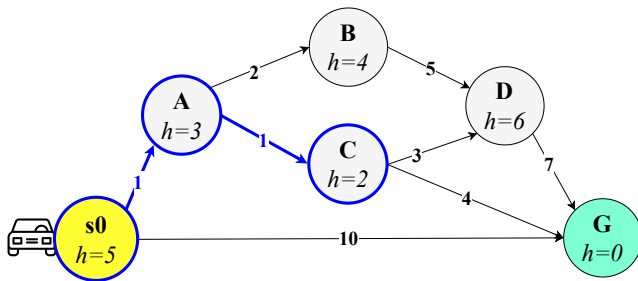
A* Search Example (3 of 5)

- $OPENSET = \langle A, G \rangle$
 - $f(A) = g(A) + h(A) = 1 + 3 = 4$
 - $f(G) = g(G) + h(G) = 10 + 0 = 10$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: **A**
 - **Add successors** of **A** in $OPENSET$:
 - $OPENSET = \langle \underline{C}, \underline{B}, G \rangle$



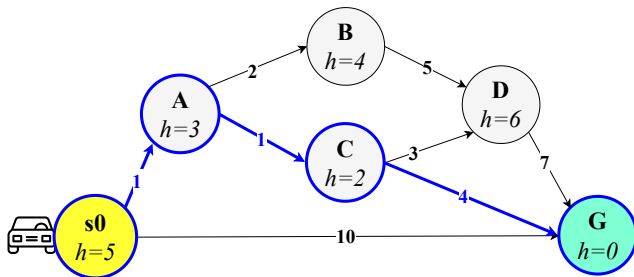
A* Search Example (4 of 5)

- $OPENSET = \langle C, B, G \rangle$
 - $f(C) = g(C) + h(C) = 2 + 2 = 4$
 - $f(B) = g(B) + h(B) = 3 + 4 = 7$
 - $f(G) = g(G) + h(G) = 10 + 0 = 10$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: **C**
 - **Add successors** of **C** in $OPENSET$:
 - $OPENSET = \langle \underline{G}, B, \underline{D} \rangle$



A* Search Example (5 of 5)

- $OPENSET = \langle G, B, D \rangle$
 - $f(G) = g(G) + h(G) = 6 + 0 = \mathbf{6}$
 - $f(B) = g(B) + h(B) = 3 + 4 = \mathbf{7}$
 - $f(D) = g(D) + h(D) = 5 + 6 = \mathbf{11}$
 - **Remove** from $OPENSET$ the state with **minimum** $f(s)$: G
 - G is the goal state, then return **solution**:
 - $s_0 \rightarrow_1 A \rightarrow_1 C \rightarrow_4 G$



Theoretical Properties of A* Search

Theorem (Termination and Completeness)

A search algorithm is complete and terminate if:*

- 1 *The branching factor is finite; and*
- 2 *The action costs are non-negative.*

Theorem (Optimal Efficiency)

A search algorithm is optimal if:*

- 1 **Admissibility:** *$h(s)$ must be an admissible heuristic; and*
 - 2 **Consistency:** *h must be consistent (monotone).*
- **Space Complexity:** $O(b^d)$, where b is the **branching factor**, and d is the **depth**.

- We studied *Heuristic Search* for **deterministic** environments;
 - A* is very effective if we have a “**good**” heuristic function h .
 - Are there other Heuristic Search algorithms?
Yes, Weighted A*, IDA*, Greedy Best-First Search, others.
- Can we do *Heuristic Search* in **non-deterministic/stochastic** environments?!
 - **AND/OR Heuristic Search (AO*)**.

Non-Deterministic Heuristic Search (AO*)